Information Retrieval

Prof. Alberto Sillitti

2. Web basics

INNOPOLIS UNIVERSITY

Web content evolution

Web 1.0	Web 2.0	Web 2.0, 3.0
HTML – Subset of SGML (markup language)	**xHTML** – fusion of HTML tags and XML standard	**[x]HTML5** – valuable layout changes, semantic tags were added
SGML parsers	**XML** parsers	**HTML5/XML** parsers
For presentation in browsers	For displaying interactive and media content	For creation of web-applications that support semantic markup

HTTP

HTTP (hypertext transfer protocol) – application (7) level protocol to deliver text data. Created to transfer hypertext. Provide communication between *client* (usually browser) and *server* (web-server) using client requests and server responses.

HTTP v1.0 – does not support using single TCP session for multiple requests. Supports following client request methods:

- GET – get content from the server
- HEAD – get only header from the server without content ("what to expect")
- POST – sent data to the server

HTTP v1.1. – supports also PUT, DELETE, TRACE, OPTIONS, CONNECT, PATCH

HTTP/2 - SDPY (Google) based update. Binary. Header compression, Server pushes, conveyor requests, request multiplexing over single TCP

HTTP/3 — QUIC, binary, UDP based

HTTP header

HTTP request =
Request URI + HOST + [[headers]] + <empty_line> + body

```
POST /index.html HTTP/1.1
HOST: example.org:8080
Cache-Control: max-age=0
Accept: text/html,application/xhtml+xml
...

param1=value1&param2=value2
```

HTTP response=

Response code + headers + <empty_line> + body

```
HTTP/1.0 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 3123
...
```

Important headers (1/2)

REQUEST

`Accept, Accept-Charset, Accept-Encoding` – formats, that your browser understands (text/plain, application/xml), encodings (utf-8) and supported compression algorithms (gzip, deflate)

`Authorization` – header that stores authentication type, credentials/keys,…

```
Authorization: Basic QWxhZGRpbjpvcGVuIHNlc2FtZQ==
```

`Content-Length` – request body length (same for response)

`User-Agent` – browser and operating system

`Upgrade` – request to change communication protocol (`Upgrade:websocket`)

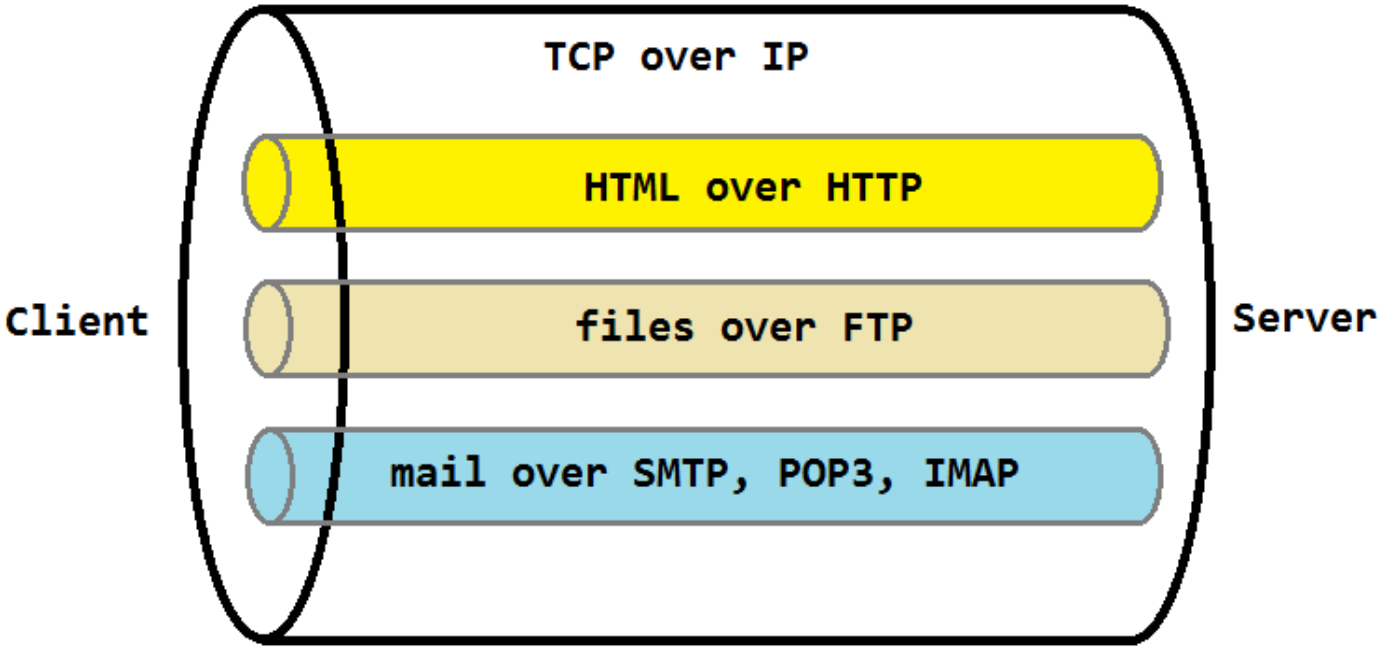
Important headers (2/2)

RESPONSE

`Cache-Control` – time to store document in a browser cache

`Content-Encoding, Content-Language, Content-Type` – content characteristics

Some other protocols



Document languages

- SGML – meta language for description of markup languages. It defines
  - Allowed symbol alphabet (SGML declaration)
  - DTD (data type definition) – markup syntax + semantics
- XML – simplified subset of SGML
  - XML Schema languages (DTD, W3C XSD)
- HTML – application of SGML (initially)
  - xHTML – application of XML

Schema languages

- **DTD, XML Schema** – define document structure and node constraints
- Used for
  - Defining semantic rules (for values, number of children…)
  - Document pre-validation
- Document can be:
  - **type-valid** – meet all DTD constraints
  - **tag-valid** – meet all [SGML/XML] **tag** constraints

DTD example

```
<!ELEMENT people_list (person*)>
<!ELEMENT person (name, birthdate?, gender?, socialsecuritynumber?)>
<!ELEMENT name (#PCDATA) >
<!ELEMENT birthdate (#PCDATA) >
<!ELEMENT gender (#PCDATA) >
<!ELEMENT socialsecuritynumber (#PCDATA) >
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE people_list SYSTEM "example.dtd">
<people_list>
   <person>
      <name>
         Fred Bloggs
      </name>
      <birthdate>
         27/11/2008
      </birthdate>
      <gender>
         Male
      </gender>
      <socialsecuritynumber>
         1234567890
      </socialsecuritynumber>
   </person>
</people_list>
```

```
<!-- Валидация простого HTML 4.01 -->
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
   "http://www.w3.org/TR/html4/strict.dtd">
```

XSD example

```xml
<?xml version="1.0" encoding="utf-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="country">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="country_name" type="xs:string"/>
        <xs:element name="population" type="xs:decimal"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```xml
<?xml version="1.0" encoding="utf-8"?>
<country>
    <country_name>France</country_name>
    <population>59.7</population>
</country>
```

HTML5 example

```
1  <!doctype html>
2  <html>
3      <head>
4          <title>my title</title>
5      </head>
6      <body>
7          body
8          <footer>
9          <!-- html5 specific -->
10         footer
11         </footer>
12     </body>
13 </html>
```

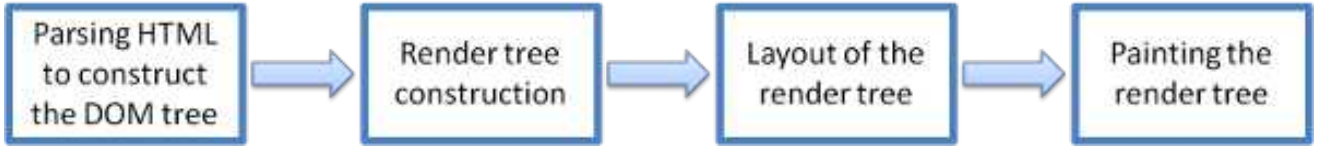
*ML document parsing (1/2)

- **SAX** (Simple API for XML)
  - Raises an event/error when new element (token) appears (considering document as stream of tokens and errors)
  - Works either as
    - callback-methods (push) or
    - cursor (pull, StAX)
  - Requires constant memory
    - Good for embedded systems
  - Does not know anything about document's model

*ML document parsing (2/2)

- **DOM** (**Document object model**, DOM tree)
  - Creates full document model
  - Used in browsers
  - Unpredictable memory usage
    - XML-bombs using DTD
  - Query languages (CSS-selectors, xpath, xquery)

Browsers



- Good article about browser architecture
- Browser Layout Engine (html + css)
  - ~~**Trident**~~ (IE), "~~Edge~~" (Spartan) → Chromium (2019)
  - **Gecko** (Mozilla)
  - ~~**WebKit**~~ (Safari, Chromium-family), WebCore
    - **Blink** (Chrome 28+, Opera 15+, Chrome for Android)
  - Others (KHTML, ~~Presto~~)

HTML tags (1/2)

- Tag
  - **tag name** from HTML Schema - mandatory
    - `<td>..</td>`
  - **Closing tag** - mandatory
    - `<div> …. </div>`
    - `<img src="kitty.png" />`
  - Attributes – define semantics
    -
```
<div id="div1"
    style="border:1px"
    class="myDiv">
</div>
```

HTML tags (2/2)

- Can have **inner tags** or **inner content** (text)
  - `<script> console.log(text); </script>`
  - ```

         … 
        <input />

    ```
- Layout of the tag is defined by the **style**
  - **In attribute**
  - **In CSS specification**
  - **By default**
    ```

Style

- inline-styles
  - ** **
- styles inside document
  - **<head><style>** …… **</style></head>**
- styles in separate CSS file
  - <link rel="stylesheet" type="text/css" **href="xxx.css"/>**

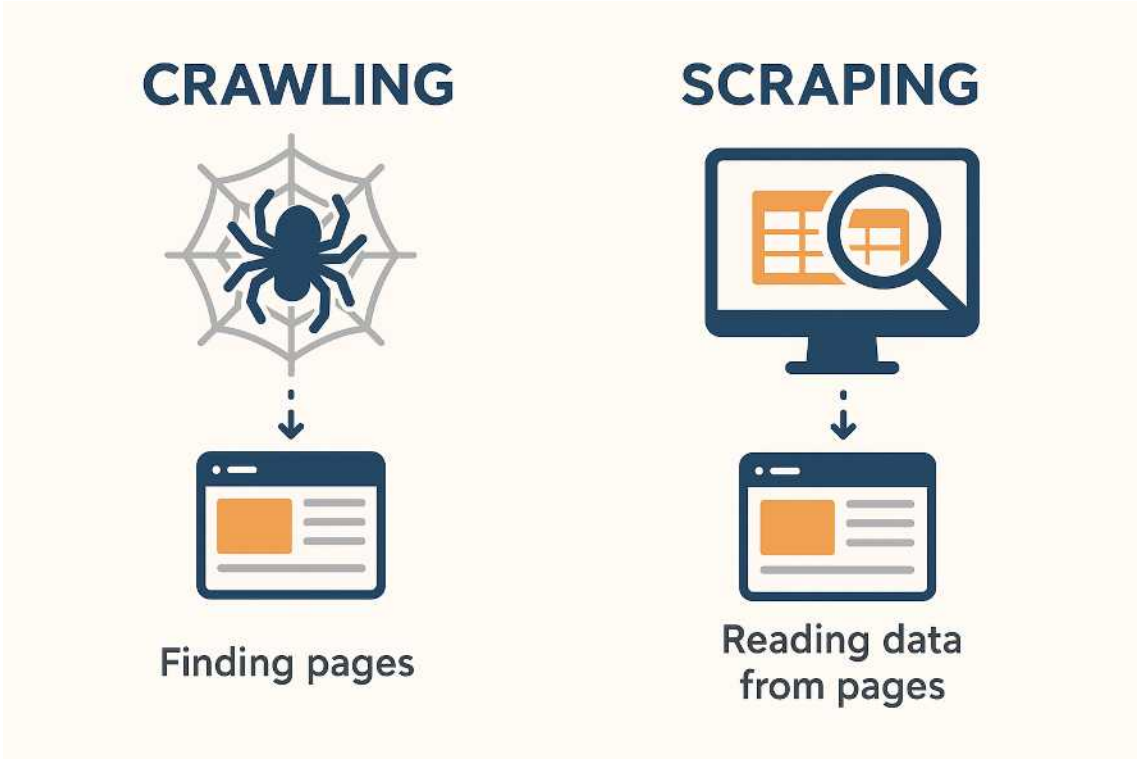
Style syntax

```
some css selector1
{
  property1: value1 ;
  property2: value2 ;
}
some css selector2
{
  property3: value3 ;
  property4: value4 ;
}
```

Fingerprinting (FingerprintJS)

- `navigator.userAgent, navigator.language`
- `new Date().getTimezoneOffset()`
- `screen.height, screen.width, screen.colorDepth`
- HTML5 features support (yes/no)
- `doNotTrack/DNT` flag (ЫЫЫЫЫ), `cpuClass, platform`
- Installed extensions
- canvas fingerprint (draw on canvas and `toDataURL()`) — fonts depend on platform
- WebGL fingerprint (for iOS)
- Installed fonts

Crawling and scraping



How to do it?

You need a browser engine + JS

1. [Headless] browsers
2. Drivers to manage browsers
3. Automation software:
    a. Selenium
    b. Puppeteer
    c. ...

How to do it?

- How would you parse all links from the Wikipedia article page?
- ```
  import requests
  from bs4 import BeautifulSoup
  …
  ```
- **Stop here**! Every big **company knows** that you will parse it's data. It wants to minimize harm you can do. **APIs**! Free anonymous, free authenticated, paid.

1. Wikipedia API
2. VK API
3. Yandex Search API
4. Google Open Search API
5. ...
  ```

Problem: I was downloading... but it stopped working

1. Company **considers** the data (and service) it has as:
    a. A property
    b. A competitive advantage
2. Thus, company **protects** its data from grabbing (and services from proxying):
    a. With API regulations (Ya, ...) — what is the allowed rate
    b. Etiquette (Wiki)
    c. Access keys to control grabbing and proxying speed
    d. Special legal statements that prohibit grabbing (ASOS)

Problem: I was downloading… but it stopped working

3. To **enforce** you to obey
    a. Access key restriction
    b. IP [range] blocking
    c. Browser **fingerprint** blocking
4. So, to speed or just enable you crawling …

Allows and disallows

- robots.txt **prohibits**
  - http://innopolis.ru/robots.txt
  - https://yandex.ru/robots.txt
  - Wiki :)

```
# Sorry, wget in its recursive mode is a frequent problem.
# Please read the man page and use it properly; there is a
# --wait option you can use to set the delay between hits,
# for instance.
#
User-agent: wget
Disallow: /
```

- sitemap.xml **helps.** Start with robots : https://www.kinopoisk.ru/robots.txt
  - Sitemap: https://www.kinopoisk.ru/sitemaps/sitemap.xml